

Perform Incremental Map-Reduce

NOTE

Aggregation Pipeline as Alternative

Starting in MongoDB 5.0, map-reduce is deprecated:

- Instead of map-reduce, you should use an [aggregation pipeline](#). Aggregation pipelines provide better performance and usability than map-reduce.
- You can rewrite map-reduce operations using aggregation pipeline stages, such as `$group`, `$merge`, and others.
- For map-reduce operations that require custom functionality, you can use the `$accumulator` and `$function` aggregation operators. You can use those operators to define custom aggregation expressions in JavaScript.

For examples of aggregation pipeline alternatives to map-reduce, see:

- [Map-Reduce to Aggregation Pipeline](#)
- [Map-Reduce Examples](#)

This section has an example aggregation pipeline alternative to map-reduce that does not use a custom function. For an example that uses a custom function, see [Map-Reduce to Aggregation Pipeline](#).

To perform map-reduce operations, MongoDB provides the `mapReduce` command and, in `mongosh`, the `db.collection.mapReduce()` wrapper method.

If the map-reduce data set is constantly growing, you may want to perform an incremental map-reduce rather than performing the map-reduce operation over the entire data set each time.

To perform incremental map-reduce:

1. Run a map-reduce job over the current collection and output the result to a separate collection.
2. When you have more data to process, run subsequent map-reduce jobs with:
 - the `query` parameter that specifies conditions that match *only* the new documents.
 - the `out` parameter that specifies the `reduce` action to merge the new results into the existing output collection.

Consider the following example where you schedule a map-reduce operation on a `usersessions` collection to run at the end of each day.

Data Setup

The `usersessions` collection contains documents that log users' sessions each day, for example:

```
db.usersessions.insertMany([
  { userid: "a", start: ISODate('2020-03-03 14:17:00'), length: 95 },
  { userid: "b", start: ISODate('2020-03-03 14:23:00'), length: 110 },
  { userid: "c", start: ISODate('2020-03-03 15:02:00'), length: 120 },
  { userid: "d", start: ISODate('2020-03-03 16:45:00'), length: 45 },
  { userid: "a", start: ISODate('2020-03-04 11:05:00'), length: 105 },
  { userid: "b", start: ISODate('2020-03-04 13:14:00'), length: 120 },
  { userid: "c", start: ISODate('2020-03-04 17:00:00'), length: 130 },
  { userid: "d", start: ISODate('2020-03-04 15:37:00'), length: 65 }
])
```

Initial Map-Reduce of Current Collection

Run the first map-reduce operation as follows:

1. Define the map function that maps the `userid` to an object that contains the fields `total_time`, `count`, and `avg_time`:

```
var mapFunction = function() {
  var key = this.userid;
  var value = { total_time: this.length, count: 1, avg_time: 0 };
  emit( key, value );
};
```

2. Define the corresponding reduce function with two arguments `key` and `values` to calculate the total time and the count. The `key` corresponds to the `userid`, and the `values` is an array whose elements corresponds to the individual objects mapped to the `userid` in the `mapFunction`.

```
var reduceFunction = function(key, values) {
  var reducedObject = { total_time: 0, count: 0, avg_time: 0 };
  values.forEach(function(value) {
    reducedObject.total_time += value.total_time;
    reducedObject.count += value.count;
  });
  return reducedObject;
};
```

3. Define the finalize function with two arguments `key` and `reducedValue`. The function modifies the `reducedValue` document to add another field `average` and returns the modified document.

```
var finalizeFunction = function(key, reducedValue) {
  if (reducedValue.count > 0)
    reducedValue.avg_time = reducedValue.total_time /
    reducedValue.count;
  return reducedValue;
};
```

4. Perform map-reduce on the `usersessions` collection using the `mapFunction`, the `reduceFunction`, and the `finalizeFunction` functions. Output the results to a

collection `session_stats`. If the `session_stats` collection already exists, the operation will replace the contents:

```
db.usersessions.mapReduce(  
  mapFunction,  
  reduceFunction,  
  {  
    out: "session_stats",  
    finalize: finalizeFunction  
  }  
)
```

5. Query the `session_stats` collection to verify the results:

```
db.session_stats.find().sort( { _id: 1 } )
```

The operation returns the following documents:

```
{ "_id" : "a", "value" : { "total_time" : 200, "count" : 2, "avg_time" :  
100 } }  
{ "_id" : "b", "value" : { "total_time" : 230, "count" : 2, "avg_time" :  
115 } }  
{ "_id" : "c", "value" : { "total_time" : 250, "count" : 2, "avg_time" :  
125 } }  
{ "_id" : "d", "value" : { "total_time" : 110, "count" : 2, "avg_time" :  
55 } }
```

Subsequent Incremental Map-Reduce

Later, as the `usersessions` collection grows, you can run additional map-reduce operations. For example, add new documents to the `usersessions` collection:

```
db.usersessions.insertMany([  
  { userid: "a", ts: ISODate('2020-03-05 14:17:00'), length: 130 },  
  { userid: "b", ts: ISODate('2020-03-05 14:23:00'), length: 40 },  
  { userid: "c", ts: ISODate('2020-03-05 15:02:00'), length: 110 },  
  { userid: "d", ts: ISODate('2020-03-05 16:45:00'), length: 100 }  
)
```

At the end of the day, perform incremental map-reduce on the `usersessions` collection, but use the `query` field to select only the new documents. Output the results to the collection `session_stats`, but `reduce` the contents with the results of the incremental map-reduce:

```
db.usersessions.mapReduce(  
  mapFunction,  
  reduceFunction,  
  {  
    query: { ts: { $gte: ISODate('2020-03-05 00:00:00') } },  
    out: { reduce: "session_stats" },  
    finalize: finalizeFunction  
  }
```

```
}  
);
```

Query the `session_stats` collection to verify the results:

```
db.session_stats.find().sort( { _id: 1 } )
```

The operation returns the following documents:

```
{ "_id" : "a", "value" : { "total_time" : 330, "count" : 3, "avg_time" : 110 } }  
{ "_id" : "b", "value" : { "total_time" : 270, "count" : 3, "avg_time" : 90 } }  
{ "_id" : "c", "value" : { "total_time" : 360, "count" : 3, "avg_time" : 120 } }  
{ "_id" : "d", "value" : { "total_time" : 210, "count" : 3, "avg_time" : 70 } }
```

Aggregation Alternative

As an alternative to map-reduce, you can use an aggregation pipeline that combines `$group` and `$merge` stages to achieve the same result in a more flexible operation.

Recreate the `usersessions` collection:

```
db.usersessions.drop();  
db.usersessions.insertMany([  
  { userid: "a", start: ISODate('2020-03-03 14:17:00'), length: 95 },  
  { userid: "b", start: ISODate('2020-03-03 14:23:00'), length: 110 },  
  { userid: "c", start: ISODate('2020-03-03 15:02:00'), length: 120 },  
  { userid: "d", start: ISODate('2020-03-03 16:45:00'), length: 45 },  
  { userid: "a", start: ISODate('2020-03-04 11:05:00'), length: 105 },  
  { userid: "b", start: ISODate('2020-03-04 13:14:00'), length: 120 },  
  { userid: "c", start: ISODate('2020-03-04 17:00:00'), length: 130 },  
  { userid: "d", start: ISODate('2020-03-04 15:37:00'), length: 65 }  
)
```

Using the available aggregation pipeline operators, you can rewrite the map-reduce example without defining custom functions:

```
db.usersessions.aggregate([  
  { $group: { _id: "$userid", total_time: { $sum: "$length" }, count: { $sum: 1 }, avg_time: { $avg: "$length" } } },  
  { $project: { value: { total_time: "$total_time", count: "$count", avg_time: "$avg_time" } } },  
  { $merge: {  
    into: "session_stats_agg",  
    whenMatched: [ { $set: {  
      "value.total_time": { $add: [ "$value.total_time",  
        "$$new.value.total_time" ] },
```

```

"value.count": { $add: [ "$value.count", "$$new.value.count" ] },
"value.avg": { $divide: [ { $add: [ "$value.total_time",
"$$new.value.total_time" ] }, { $add: [ "$value.count",
"$$new.value.count" ] } ] }
} } ],
whenNotMatched: "insert"
}}
])

```

1. The `$group` groups by the `userid` and calculates:

- The `total_time` using the `$sum` operator
- The `count` using the `$sum` operator
- The `avg_time` using the `$avg` operator

The operation returns the following documents:

```

{ "_id" : "c", "total_time" : 250, "count" : 2, "avg_time" : 125 }
{ "_id" : "d", "total_time" : 110, "count" : 2, "avg_time" : 55 }
{ "_id" : "a", "total_time" : 200, "count" : 2, "avg_time" : 100 }
{ "_id" : "b", "total_time" : 230, "count" : 2, "avg_time" : 115 }

```

2. The `$project` stage reshapes the output document to mirror the map-reduce's output to have two fields `_id` and `value`. The stage is optional if you do not need to mirror the `_id` and `value` structure.

```

{ "_id" : "a", "value" : { "total_time" : 200, "count" : 2, "avg_time" : 100 } }
{ "_id" : "d", "value" : { "total_time" : 110, "count" : 2, "avg_time" : 55 } }
{ "_id" : "b", "value" : { "total_time" : 230, "count" : 2, "avg_time" : 115 } }
{ "_id" : "c", "value" : { "total_time" : 250, "count" : 2, "avg_time" : 125 } }

```

3. The `$merge` stage outputs the results to a `session_stats_agg` collection. If an existing document has the same `_id` as the new result, the operation applies the specified pipeline to calculate the `total_time`, `count`, and `avg_time` from the result and the existing document. If there is no existing document with the same `_id` in the `session_stats_agg`, the operation inserts the document.

4. Query the `session_stats_agg` collection to verify the results:

```
db.session_stats_agg.find().sort( { _id: 1 } )
```



The operation returns the following documents:

```

{ "_id" : "a", "value" : { "total_time" : 200, "count" : 2, "avg_time" : 100 } }

```

```
{ "_id" : "b", "value" : { "total_time" : 230, "count" : 2, "avg_time" : 115 } }
{ "_id" : "c", "value" : { "total_time" : 250, "count" : 2, "avg_time" : 125 } }
{ "_id" : "d", "value" : { "total_time" : 110, "count" : 2, "avg_time" : 55 } }
```

5. Add new documents to the `usersessions` collection:

```
db.usersessions.insertMany([
  { userid: "a", ts: ISODate('2020-03-05 14:17:00'), length: 130 },
  { userid: "b", ts: ISODate('2020-03-05 14:23:00'), length: 40 },
  { userid: "c", ts: ISODate('2020-03-05 15:02:00'), length: 110 },
  { userid: "d", ts: ISODate('2020-03-05 16:45:00'), length: 100 }
])
```

6. Add a `$match` stage at the start of the pipeline to specify the date filter:

```
db.usersessions.aggregate([
  { $match: { ts: { $gte: ISODate('2020-03-05 00:00:00') } } },
  { $group: { _id: "$userid", total_time: { $sum: "$length" }, count: { $sum: 1 }, avg_time: { $avg: "$length" } } },
  { $project: { value: { total_time: "$total_time", count: "$count", avg_time: "$avg_time" } } },
  { $merge: {
    into: "session_stats_agg",
    whenMatched: [ { $set: {
      "value.total_time": { $add: [ "$value.total_time",
        "$$new.value.total_time" ] },
      "value.count": { $add: [ "$value.count", "$$new.value.count" ] },
      "value.avg_time": { $divide: [ { $add: [ "$value.total_time",
        "$$new.value.total_time" ] }, { $add: [ "$value.count",
        "$$new.value.count" ] } ] } }
    ] },
    whenNotMatched: "insert"
  } } ],
  {}
])
```

7. Query the `session_stats_agg` collection to verify the results:

```
db.session_stats_agg.find().sort( { _id: 1 } )
```

The operation returns the following documents:

```
{ "_id" : "a", "value" : { "total_time" : 330, "count" : 3, "avg_time" : 110 } }
{ "_id" : "b", "value" : { "total_time" : 270, "count" : 3, "avg_time" : 90 } }
{ "_id" : "c", "value" : { "total_time" : 360, "count" : 3, "avg_time" : 120 } }
{ "_id" : "d", "value" : { "total_time" : 210, "count" : 3, "avg_time" : 70 } }
```

8. Optional. To avoid having to modify the aggregation pipeline's `$match` date condition each time you run, you can define wrap the aggregation in a helper function:

```
updateSessionStats = function(startDate) {
  db.usersessions.aggregate([
    { $match: { ts: { $gte: startDate } } },
    { $group: { _id: "$userid", total_time: { $sum: "$length" }, count:
    { $sum: 1 }, avg_time: { $avg: "$length" } } },
    { $project: { value: { total_time: "$total_time", count: "$count",
    avg_time: "$avg_time" } } },
    { $merge: {
      into: "session_stats_agg",
      whenMatched: [ { $set: {
        "value.total_time": { $add: [ "$value.total_time",
        "$$new.value.total_time" ] },
        "value.count": { $add: [ "$value.count", "$$new.value.count" ] },
        "value.avg_time": { $divide: [ { $add: [ "$value.total_time",
        "$$new.value.total_time" ] }, { $add: [ "$value.count",
        "$$new.value.count" ] } ] } }
      } } ],
      whenNotMatched: "insert"
    } } ],
  });
};
```



Then, to run, you would just pass in the start date to the `updateSessionStats()` function:

```
updateSessionStats(ISODate('2020-03-05 00:00:00'))
```



TIP

See also:

- [\\$merge Examples](#)
- [On-Demand Materialized Views](#)
- [Map-Reduce to Aggregation Pipeline](#)